BACHELOR OF TECHNOLOGY · COMPUTER SCIENCE · 2025 – 2026

Final Year Project Synopsis

LIBRARY MANAGEMENT SYSTEM

A Full-Stack RESTful Application · Spring Boot 2.7 · Java 8 · MySQL 8

Java 8 | Spring Boot | Spring Security | MySQL 8

JWT / RBAC | Flyway | Swagger UI | REST API

Project Title	Library Management System
Technology	Java 8 · Spring Boot 2.7.18 · MySQL 8 · REST API
Academic Year	2025 – 2026
Guide	[Guide Name, Designation]
Student	[Student Name] · Roll No: [Roll Number]
Department	[Department Name]
Institution	[College / University Name]
Date	May 21, 2026

TABLE OF CONTENTS

1. PROJECT TITLE

Library Management System (LMS)

A full-stack, RESTful web application for automating all library operations including book cataloguing, member management, loan tracking, fine calculation, reservation queuing, and real-time notifications — built on **Spring Boot 2.7.18** with **MySQL 8**.

2. INTRODUCTION

Libraries are the backbone of academic institutions, research organizations, and public communities. Managing vast collections manually is time-consuming and error-prone. A **Library Management System (LMS)** replaces paper-based record keeping, manual fine calculations, and physical reservation queues with a centralized, role-based digital platform accessible from any device.

This project presents a production-grade LMS developed using **Java 8** and **Spring Boot 2.7.18** as the backend framework, with **MySQL 8** as the relational database. The system exposes a fully documented **RESTful API** (Swagger UI at `/swagger-ui.html`) that can power any frontend — a web browser, Android application, or desktop client.

Key Highlights

- Stateless **JWT-based authentication** with access tokens (15 min) and refresh tokens (7 days)
- **Role-Based Access Control (RBAC)** with three roles: Admin, Librarian, and Member
- **Fine policy engine** that auto-generates overdue fines based on membership tier
- **Reservation queue** with automated notification when a book becomes available
- **Scheduled background jobs** for due-date reminders and overdue alerts
- **In-memory caching** for high-frequency read operations (book listings)
- Full **Flyway database migration** support for version-controlled schema changes
- **Swagger / OpenAPI 3** documentation for every endpoint

3. PROBLEM STATEMENT

Traditional library systems suffer from the following critical limitations:

#	Problem	Impact
1	Manual Record Keeping	Issue and return records maintained in physical registers lead to frequent data-entry errors, lost records, and difficulty in audit.
2	No Real-Time Availability	Members cannot check book availability without physically visiting the library, resulting in wasted trips.
3	Inefficient Fine Management	Overdue fines are calculated manually, often inconsistently, and members have no transparent view of their outstanding dues.
4	No Reservation System	When a book is checked out, interested members cannot formally reserve it — leading to repeated visits and frustration.
5	Lack of Reporting	Librarians and administrators have no easy way to generate reports on overdue books, active loans, popular titles, or member activity.

6	Security Risks	Paper records have no access control; anyone can tamper with registers. There is no audit trail.
7	Scalability Issues	As collections and member bases grow, manual management becomes completely unworkable.
8	No Notification System	Members receive no reminders about upcoming due dates, leading to high overdue rates and revenue loss.

4. OBJECTIVES OF THE PROJECT

1. Automate Book Cataloguing — Provide a complete CRUD interface for managing books, authors, publishers, categories, and physical book copies (with barcodes and shelf locations).

2. Member Lifecycle Management — Register members, assign membership types (Standard / Premium / Student), track membership expiry, and manage member status.

3. Streamline Loan Operations — Enable librarians to issue and return books digitally, enforce loan limits based on membership tier, and support loan renewals.

4. Automated Fine Calculation — Calculate overdue fines automatically upon book return, applying membership-specific daily rates, grace periods, and maximum fine caps.

5. Reservation Queue Management — Allow members to place holds on unavailable books and automatically notify the next member in queue when the book is returned.

6. Role-Based Security — Implement JWT-based stateless authentication with three roles (Admin, Librarian, Member), each with precisely defined permissions.

7. Scheduled Notifications — Send automated due-date reminders 2 days before and overdue alerts the day after the due date via an internal notification store.

8. Reporting and Analytics — Provide reports on overdue loans, fine collection, top-borrowed books, and member activity.

9. API Documentation — Expose a fully documented REST API via Swagger UI for frontend developers and third-party integrations.

10. Maintainability — Use Flyway migrations to version-control the database schema, ensuring repeatable deployments.

5. SCOPE OF THE PROJECT

In Scope

Area	Details
Book Management	Add, update, archive books; manage multiple physical copies (barcodes, shelf codes)
Author & Publisher	Full CRUD for authors (biography, nationality) and publishers (contact details)
Category & Shelf	Hierarchical categories, shelf assignment with floor/section/capacity
Member Management	Register, update, suspend, delete members; track membership type and expiry
Staff Management	Manage librarian accounts with employee codes and role assignments
Loan Operations	Issue, return, and renew book copies; enforce business rules per membership tier
Fine System	Auto-generate overdue fines; support partial payment, full payment, and fine waiver

Reservations	Place, cancel, and manage holds; priority-queue; auto-notify on availability
Notifications	Due reminders (2 days before), overdue alerts, reservation-ready notices
Reports	Overdue loans report, fine collection summary, loan activity, member reports
Security	JWT access + refresh tokens; BCrypt password hashing; role + permission RBAC
API Docs	Swagger UI with all endpoints documented (request/response schemas)

Out of Scope

- Payment gateway integration (fine payments are recorded but not processed online)
- Digital book (e-book / PDF) content hosting
- Multi-library / multi-branch management
- Mobile application (the API is mobile-ready, but the app itself is out of scope)
- Email/SMS delivery (notification records are created; actual email sending requires SMTP config)

6. EXISTING SYSTEM

Manual / Legacy Library Systems

Most small and medium libraries still operate with manual processes: physical accession registers, paper issue cards, manual fine calculation, and card catalogue browsing.

Problem	Impact
Data redundancy in paper records	Inaccurate stock counts, duplicate entries
No real-time availability check	Wasted member trips to the library
Manual fine calculation	Inconsistent fines, revenue loss
No audit trail	Cannot trace who issued or returned a book if records are tampered
No reservation mechanism	Members lose interest if books are unavailable
Reporting is labor-intensive	Weeks to compile annual reports
No automated reminders	High overdue rates

Partially-Automated Systems (Older Software)

Some libraries use older desktop applications (e.g., LibraryWorld, OpenBiblio) that are not web-accessible (no REST API), not scalable beyond a single machine, missing modern security features (no JWT, no RBAC), and difficult to customize or extend.

7. PROPOSED SYSTEM

The proposed **Library Management System** is a cloud-deployable, RESTful web service that eliminates all the drawbacks of existing systems by following a **three-tier architecture**:

Tier	Component	Technology

Presentation Tier	Any REST client (browser app, Postman, mobile app, Swagger UI)	HTTP/JSON
Business Logic Tier	Spring Boot 2.7.18 application server	Java 8
Data Tier	MySQL 8 relational database	MySQL 8 / HikariCP

Key Features of the Proposed System

- **Centralized Digital Catalogue** — All books, copies, authors, and categories stored in a normalized relational database.
- **Stateless JWT Authentication** — No session state; scales horizontally. Access tokens expire in 15 min; refresh tokens in 7 days.
- **Three-Tier RBAC** — Admin (full access), Librarian (operations), Member (self-service). Permissions are granular.
- **Automated Fine Engine** — On book return, the system calculates overdue days minus grace period, applies the membership-specific daily rate, and caps at the maximum fine.
- **Reservation Queue** — When a member places a hold, a queue position is assigned. When the book is returned, the next member in queue is automatically notified.
- **Scheduled Jobs** — Spring @Scheduled tasks run at 9 AM (due reminders) and 8 AM (overdue alerts) daily.
- **Caching** — The book list endpoint is cached in-memory, reducing database round-trips.
- **Swagger Documentation** — Every API endpoint documented with request/response schemas at /swagger-ui.html.

8. TECHNOLOGIES USED

Backend

Technology	Version	Purpose
Java	8 (JDK 1.8)	Core programming language
Spring Boot	2.7.18	Application framework, auto-configuration
Spring Web MVC	5.3.x	REST controller layer
Spring Data JPA	2.7.x	ORM layer (Hibernate dialect)
Spring Security	5.7.x	Authentication & authorization framework
Spring Cache	2.7.x	Declarative caching (@Cacheable)
Spring Scheduling	2.7.x	Cron-based background jobs (@Scheduled)
Hibernate	5.6.x	JPA implementation, MySQL8 dialect
JJWT	0.9.1	JWT generation and validation
Lombok	1.18.30	Boilerplate reduction (@Data, @Builder)
MapStruct	1.5.5.Final	Entity-to-DTO mapping
SpringDoc OpenAPI	1.7.0	Swagger UI + OpenAPI 3 documentation
Bean Validation	2.0	Request DTO validation (@Valid, @NotBlank)

Database & Build Tools

Tool	Version	Purpose
MySQL	8.0	Primary relational database
HikariCP	4.x	High-performance JDBC connection pool
Flyway	Managed	Database migration versioning
Apache Maven	3.9.6	Build automation, dependency management
IntelliJ IDEA	—	IDE for Java development
Postman	v10+	API testing
MySQL Workbench	—	Database design and query execution
Git	—	Version control

9. SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

Component	Minimum	Recommended
Operating System	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04
JDK	Java 8 (JDK 1.8.0_202+)	Java 8 (latest patch)
MySQL Server	8.0	8.0.33+
Apache Maven	3.6+	3.9.6
Web Browser	Chrome 90+ / Firefox 88+	Chrome (latest)
IDE	IntelliJ IDEA Community / Eclipse	IntelliJ IDEA Ultimate
Postman	Any version	v10+

Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD equivalent	Intel Core i5 or better
RAM	4 GB	8 GB
Storage	10 GB free disk space	20 GB SSD
Network	—	LAN/Wi-Fi for multi-user access
Display	1280×720	1920×1080

10. SYSTEM ARCHITECTURE

The system follows a layered architecture with clear separation of concerns:

Layer	Components	Responsibility
Client Layer	Browser / Postman / Mobile App / Swagger UI (:8085)	HTTP/S REST JSON requests
Security Layer	JwtAuthenticationFilter, CustomUserDetailsService, SecurityConfig	JWT validation, CSRF off, stateless

Controller Layer	AuthController, BookController, MemberController, LoanController, FineController, ReservationController, ReportController	REST endpoints /api/v1/*
Service Layer	BookServiceImpl, LoanServiceImpl, FineServiceImpl, MemberServiceImpl, AuthServiceImpl, ReservationServiceImpl	Business logic, @Transactional
Repository Layer	14 Spring Data JPA repositories	JPQL/SQL via HikariCP connection pool
Data Layer	MySQL 8.0 — library_db — 16 Tables, Indexes, Foreign Keys	Persistent storage
Cross-Cutting	@Cacheable, @Scheduled, GlobalExceptionHandler, Flyway Migrations	Caching, jobs, error handling, migrations

Request Flow

Step 1: Client sends HTTP request with Bearer token in Authorization header

Step 2: JwtAuthenticationFilter intercepts and validates the JWT

Step 3: CustomUserDetailsService loads user details (checks Staff table first, then Members)

Step 4: Spring Security populates SecurityContext with authenticated user and roles

Step 5: Request reaches the REST Controller

Step 6: Controller delegates to Service layer for business logic

Step 7: Service validates business rules, calls Repository layer for data

Step 8: Repository executes JPQL/SQL against MySQL via HikariCP connection pool

Step 9: Response DTO is built and serialized to JSON

Step 10: HTTP response returned to client

11. MODULES DESCRIPTION

Module 1: Authentication Module

Class: `AuthController / AuthServiceImpl`

Handles user login (staff + members via single endpoint), JWT access and refresh token generation, and new member self-registration.

Module 2: Book Catalogue Module

Class: `BookController / BookServiceImpl`

Full CRUD for books, authors, publishers, categories, shelves, and physical book copies. Includes search (by title, ISBN, author, category) and availability checking. Book listings are cached in-memory.

Module 3: Member Management Module

Class: `MemberController / MemberServiceImpl`

Register, update, suspend, and deactivate members. Tracks membership type (Standard/Premium/Student), membership expiry, and account status. Auto-generates unique member codes (MEM-XXXXXX).

Module 4: Loan Management Module

Class: `LoanController / LoanServiceImpl`

Core business module for issuing and returning book copies. Enforces pre-issue validations (active status, membership validity, no unpaid fines, loan limit check) and calculates overdue fines on return.

Module 5: Fine Management Module

Class: `FineController / FineServiceImpl`

Auto-generates overdue fines with membership-tier-specific rates and grace periods. Supports partial and full payment, and admin/librarian fine waivers.

Module 6: Reservation Module

Class: `ReservationController / ReservationServiceImpl`

Allows members to place holds on unavailable books. Maintains a priority queue and automatically promotes the next reservation to READY (with a 3-day collection window) when the book is returned.

Module 7: Notification Module

Class: `NotificationServiceImpl / ScheduledTaskService`

Creates in-app notification records for: loan due reminders (2 days before), overdue alerts (day after due date), and reservation-ready notices. Scheduled jobs run daily via @Scheduled.

Module 8: Reporting Module

Class: `ReportController / ReportServiceImpl`

Provides administrative reports: overdue loans, fine collection summary (total issued/collected/waived), top 10 most borrowed books, and member activity.

12. DATABASE DESIGN

The database `library_db` (MySQL 8) consists of **16 tables** across five functional groups:

Access Control Tables

Table	Key Columns	Notes
roles	id, name (UNIQUE), description	Seeded: ROLE_ADMIN, ROLE_LIBRARIAN, ROLE_MEMBER
permissions	id, name (UNIQUE), resource, action, description	16 seeded permissions (BOOK_CREATE, LOAN_UPDATE, FINE_WAIVE, etc.)
role_permissions	role_id (FK), permission_id (FK)	Join table mapping roles to permissions

Book Catalogue Tables

Table	Key Columns	Notes
publishers	id, name (UNIQUE), address, email, phone, website	Publisher master data
authors	id, first_name, last_name, biography, nationality, photo_url	Indexes on first_name, last_name
categories	id, name (UNIQUE), slug, parent_id (self-ref FK)	Hierarchical categories
books	id, isbn (UNIQUE), title, edition, publisher_id, status	ENUM status: ACTIVE, ARCHIVED
book_authors	book_id, author_id	M:N join table
book_categories	book_id, category_id	M:N join table
shelves	id, shelf_code (UNIQUE), floor, section, capacity, category_id	Physical shelf locations
book_copies	id, barcode (UNIQUE), book_id, shelf_id, condition, status	ENUM status: AVAILABLE, BORROWED, RESERVED, LOST, MAINTENANCE

User & Operations Tables

Table	Key Columns	Notes
members	id, member_code (UNIQUE), email (UNIQUE), membership_type, status	membership_type: STANDARD / PREMIUM / STUDENT
staff	id, employee_code (UNIQUE), email (UNIQUE), designation, role_id	Admin and Librarian accounts
loans	id, member_id, book_copy_id, issued_by, due_date, status	Composite index on (member_id, status)
reservations	id, member_id, book_id, queue_position, status, expiry_date	ENUM status: PENDING, READY, FULFILLED, CANCELLED, EXPIRED
fines	id, member_id, loan_id, fine_type, amount, paid_amount, status	ENUM status: PENDING, PARTIAL, PAID, WAIVED
fine_policies	membership_type (UNIQUE), daily_rate, max_fine, grace_period_days	Seeded per membership tier
reviews	id, member_id, book_id, rating (1-5), status	Unique constraint on (member_id, book_id)
notifications	id, recipient_id, type, channel, subject, body, is_read	ENUM type: LOAN_DUE_REMINDER, LOAN_OVERDUE, RESERVATION_READY, FINE_ISSUED

Fine Policies (Seeded Values)

Membership	Daily Rate	Max Fine	Grace Period	Loan Days	Renewals	Max Loans
STANDARD	Rs 5.00/day	Rs 500.00	1 day	14 days	2	5

PREMIUM	Rs 2.00/day	Rs 200.00	3 days	21 days	3	10
STUDENT	Rs 1.00/day	Rs 100.00	2 days	14 days	2	5

13. ER DIAGRAM / FLOWCHART

13.1 Entity Relationship Overview

Entity	Relationships
ROLES	Linked to STAFF and MEMBERS via role_id; linked to PERMISSIONS via role_permissions join table
BOOKS	M:N with AUTHORS (book_authors); M:N with CATEGORIES (book_categories); has many BOOK_COPIES; has PUBLISHER
BOOK_COPIES	Belongs to BOOKS; assigned to SHELVES; referenced by LOANS and RESERVATIONS
MEMBERS	Has many LOANS, FINES, RESERVATIONS, REVIEWS, NOTIFICATIONS; has a ROLE
STAFF	Issues and receives LOANS (issued_by, returned_to FKs); has a ROLE
LOANS	Belongs to MEMBER and BOOK_COPY; issued/received by STAFF; generates FINES
FINES	Belongs to MEMBER and LOAN; fine rate derived from FINE_POLICIES per membership type
RESERVATIONS	Belongs to MEMBER and BOOK (not copy); has queue_position for ordered notification

13.2 Loan Lifecycle States

State	Triggered By	Next States
ACTIVE	Loan issued by librarian	RETURNED, OVERDUE, LOST
RETURNED	Book returned (on time)	Terminal
OVERDUE	Scheduled job marks past-due loans	RETURNED (with fine generated)
LOST	Admin marks copy as lost	Terminal (fine generated)

13.3 Fine State Diagram

Status	Meaning	Triggered By
PENDING	Fine issued, not yet paid	Auto on overdue return
PARTIAL	Partial payment made	Member/staff payment
PAID	Fully paid	Final payment clears balance
WAIVED	Forgiven by librarian/admin	Staff discretion

14. SOURCE CODE WITH EXPLANATION

14.1 Project Structure (Key Packages)

```
com.library.lms/
LmsApplication.java -- Spring Boot entry point
config/
SecurityConfig.java -- Spring Security + JWT filter chain
DataInitializer.java -- Startup seeding
controller/ -- REST controllers (AuthController, BookController ...)
dto/request/ -- Inbound validated DTOs (LoginRequest, BookRequest ...)
dto/response/ -- Outbound DTOs (BookResponse, LoanResponse ...)
entity/ -- JPA entities (Book, Loan, Member, Fine ...)
entity/enums/ -- 13 enum types
exception/ -- GlobalExceptionHandler, BusinessException
mapper/ -- MapStruct mappers (BookMapper, LoanMapper ...)
repository/ -- 14 Spring Data JPA repositories
security/
JwtTokenProvider.java -- JWT generation & validation
JwtAuthenticationFilter.java -- Per-request token filter
CustomUserDetailsService.java-- Loads staff OR member by email
service/impl/ -- 8 service implementations
```

14.2 Security Configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
http.csrf().disable()
.sessionManagement()
.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
.and()
.authorizeHttpRequests(auth -> auth
.antMatchers("/api/v1/auth/**", "/swagger-ui.html",
"/v3/api-docs/**").permitAll()
.anyRequest().authenticated())
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
return http.build();
}
@Bean
public PasswordEncoder passwordEncoder() {
return new BCryptPasswordEncoder(12); // cost factor 12
}
}
```

14.3 JWT Token Provider (Core)

```
@Component
public class JwtTokenProvider {
private final long accessTokenExpiry = 900_000; // 15 min
private final long refreshTokenExpiry = 604_800_000; // 7 days

public String generateAccessToken(Authentication auth) {
UserDetails principal = (UserDetails) auth.getPrincipal();
return buildToken(principal.getUsername(), accessTokenExpiry, "ACCESS");
}

private String buildToken(String subject, long expiry, String type) {
return Jwts.builder()
.claim("type", type)
.setSubject(subject) // email address
.setIssuedAt(new Date())
.setExpiration(new Date(System.currentTimeMillis() + expiry))
.signWith(SignatureAlgorithm.HS256, secretKey.getBytes())
.compact();
}
}
```

14.4 Loan Issue — Core Business Logic

```
@Transactional
@PreAuthorize("hasAnyRole('ROLE_LIBRARIAN','ROLE_ADMIN')")
public LoanResponse issueLoan(LoanRequest request) {
Member member = memberRepository.findById(request.getMemberId()).orElseThrow(...);
BookCopy copy = bookCopyRepository.findById(request.getBookCopyId()).orElseThrow(...);

// Business rule validations
if (member.getStatus() != MemberStatus.ACTIVE)
throw new BusinessException("Member account is not active");
if (member.getMembershipExpiry().isBefore(LocalDate.now()))
throw new BusinessException("Member membership has expired");
if (copy.getStatus() != BookCopyStatus.AVAILABLE)
throw new BusinessException("Book copy is not available: " + copy.getBarcode());

BigDecimal unpaidFines = fineRepository.sumUnpaidByMember(member.getId());
if (unpaidFines.compareTo(BigDecimal.ZERO) > 0)
throw new BusinessException("Member has unpaid fines: " + unpaidFines);

FinePolicy policy = finePolicyRepository.findByMembershipType(member.getMembershipType())...;
long activeLoans = loanRepository.countActiveLoansByMember(member.getId());
if (activeLoans >= policy.getMaxActiveLoans())
throw new BusinessException("Max active loans limit reached: " + policy.getMaxActiveLoans());

LocalDate dueDate = LocalDate.now().plusDays(policy.getMaxLoanDays());
Loan loan = Loan.builder().member(member).bookCopy(copy).dueDate(dueDate)
.status(LoanStatus.ACTIVE).renewalCount(0).build();
copy.setStatus(BookCopyStatus.BORROWED);
bookCopyRepository.save(copy);
return toResponse(loanRepository.save(loan));
}
```

14.5 Fine Calculation Logic

```
// Called inside returnBook() when loan is overdue
private void createOverdueFine(Loan loan) {
FinePolicy policy = finePolicyRepository
.findByMembershipType(loan.getMember().getMembershipType()).orElse(null);
if (policy == null) return;

long days = Math.max(0, loan.overdueDays() - policy.getGracePeriodDays());
if (days <= 0) return; // within grace period — no fine

// Fine = MIN(daily_rate x chargeable_days, max_fine)
BigDecimal amount = policy.getDailyRate()
.multiply(BigDecimal.valueOf(days))
.min(policy.getMaxFine());

Fine fine = Fine.builder().member(loan.getMember()).loan(loan)
.fineType(FineType.OVERDUE).amount(amount)
.paidAmount(BigDecimal.ZERO).status(FineStatus.PENDING).build();
fineRepository.save(fine);
}
```

Fine Formula: Fine = MIN(daily_rate × MAX(0, overdue_days – grace_period), max_fine)

Scenario	Overdue Days	Grace	Chargeable	Rate	Fine
STANDARD member	10	1	9	Rs 5.00	Rs 45.00
PREMIUM member	10	3	7	Rs 2.00	Rs 14.00
STUDENT member	10	2	8	Rs 1.00	Rs 8.00
STUDENT (capped)	120	2	118	Rs 1.00	Rs 100.00 (capped at max_fine)

14.6 Database Migration (Flyway)

```
-- V1__create_schema.sql (excerpt)
CREATE TABLE loans (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
member_id BIGINT NOT NULL REFERENCES members(id),
book_copy_id BIGINT NOT NULL REFERENCES book_copies(id),
issued_by BIGINT REFERENCES staff(id),
returned_to BIGINT REFERENCES staff(id),
issue_date TIMESTAMP NOT NULL,
due_date DATE NOT NULL,
return_date TIMESTAMP,
status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
renewal_count INTEGER NOT NULL DEFAULT 0,
notes TEXT
);
CREATE INDEX idx_loans_member_status ON loans(member_id, status);
CREATE INDEX idx_loans_due_date ON loans(due_date);

-- V2__seed_data.sql (excerpt)
INSERT INTO staff (employee_code, email, first_name, last_name,
password_hash, designation, role_id, is_active)
VALUES ('EMP-0001', 'admin@library.com', 'System', 'Admin',
'$2a$12$LQv3c1yqBWVHxkd0LHAkCOY...', 'Administrator',
(SELECT id FROM roles WHERE name = 'ROLE_ADMIN'), TRUE);
```

15. API INTEGRATION DETAILS

The application runs on `http://localhost:8085`. All API endpoints are prefixed with `/api/v1/`.

Authentication Flow

```
Step 1: POST /api/v1/auth/login
Body: { "email": "admin@library.com", "password": "Admin@123" }
Response: { "accessToken": "eyJ...", "refreshToken": "eyJ...",
"tokenType": "Bearer", "role": "ROLE_ADMIN" }

Step 2: Include token in every subsequent request:
Authorization: Bearer eyJ...

Step 3: When access token expires (15 min):
POST /api/v1/auth/refresh
Body: { "refreshToken": "eyJ..." }
Response: { "accessToken": "eyJ..." } (new token)
```

API Endpoint Reference

Method	Endpoint	Role	Description
POST	/api/v1/auth/login	Public	Login — Staff or Member
POST	/api/v1/auth/register	Public	Register new member
POST	/api/v1/auth/refresh	Public	Refresh access token
GET	/api/v1/books	Any	List all books (paginated)
GET	/api/v1/books/search	Any	Search books (q, isbn, author, category)
POST	/api/v1/books	Admin / Librarian	Add new book
PUT	/api/v1/books/{id}	Admin / Librarian	Update book
DELETE	/api/v1/books/{id}	Admin / Librarian	Archive book (soft delete)
GET	/api/v1/books/{id}/availability	Any	Get available copy count
GET	/api/v1/members	Admin / Librarian	List all members
POST	/api/v1/members	Admin / Librarian	Create member
POST	/api/v1/loans	Admin / Librarian	Issue book copy
POST	/api/v1/loans/{id}/return	Admin / Librarian	Return book copy
POST	/api/v1/loans/{id}/renew	Any	Renew loan (subject to rules)
GET	/api/v1/loans/overdue	Admin / Librarian	Get all overdue loans
GET	/api/v1/fines	Any	Get fines (filtered by role)
POST	/api/v1/fines/{id}/pay	Any	Pay fine
POST	/api/v1/fines/{id}/waive	Admin / Librarian	Waive fine
POST	/api/v1/reservations	Member	Place reservation
DELETE	/api/v1/reservations/{id}	Member	Cancel reservation

GET	/api/v1/reports/overdue	Admin / Librarian	Overdue loans report
GET	/api/v1/reports/fines	Admin / Librarian	Fine collection summary
GET	/api/v1/reports/popular-books	Admin / Librarian	Top 10 most borrowed books

16. AUTHENTICATION AND AUTHORIZATION MODULE

Role-Permission Matrix

Permission	ROLE_ADMIN	ROLE_LIBRARIAN	ROLE_MEMBER
BOOK_CREATE	Yes	Yes	—
BOOK_READ	Yes	Yes	Yes
BOOK_UPDATE	Yes	Yes	—
BOOK_DELETE	Yes	—	—
MEMBER_CREATE	Yes	Yes	—
MEMBER_READ	Yes	Yes	—
MEMBER_UPDATE	Yes	Yes	—
MEMBER_DELETE	Yes	—	—
LOAN_CREATE	Yes	Yes	—
LOAN_READ	Yes	Yes	Own loans
LOAN_UPDATE	Yes	Yes	Own loans
FINE_READ	Yes	Yes	Own fines
FINE_WAIVE	Yes	Yes	—
REPORT_READ	Yes	Yes	—
RESERVATION_CREATE	Yes	Yes	Yes
RESERVATION_READ	Yes	Yes	Yes

JWT Token Specification

Algorithm	HS256 (HMAC-SHA256)
Secret	Configured via JWT_SECRET environment variable
Access token expiry	900,000 ms (15 minutes)
Refresh token expiry	604,800,000 ms (7 days)
Claims	sub (email), type (ACCESS/REFRESH), iat, exp
Password hashing	BCrypt with cost factor 12 — stored as 60-char hash

17. ADMIN DASHBOARD FEATURES

The Admin role (`ROLE_ADMIN`) has access to all features in the system:

Feature Area	Capabilities
Book Management	Add/update/archive books with full metadata; manage physical copies (barcodes, shelves, condition); search and filter catalogue
Member Management	Register members; assign membership types; view loan history and outstanding fines; suspend/deactivate accounts; update expiry dates
Staff Management	View all staff accounts; grant/revoke Librarian role
Loan Operations	Issue and return books; view all active/overdue/historical loans; generate overdue loans report
Fine Management	View all outstanding fines system-wide; waive fines (with staff attribution); record partial and full payments
Reports	Overdue loans with member details; fine collection summary; top 10 most borrowed books; member activity report
System Config	Fine policies (daily rates, grace periods, loan limits) stored in fine_policies table — editable by admin

18. BOOK ISSUE / RETURN MANAGEMENT

Book Issue Process

1. Librarian/Admin authenticates and obtains JWT
2. Calls POST /api/v1/loans with {memberId, bookCopyId}
3. System validates: member exists and status = ACTIVE; membership not expired; book copy status = AVAILABLE; member has no unpaid fines; active loan count < policy.maxActiveLoans
4. System determines due_date = today + policy.maxLoanDays
5. Loan record created with status = ACTIVE
6. BookCopy status updated to BORROWED
7. LoanResponse returned with all details including due_date

Book Return Process

1. Librarian/Admin calls POST /api/v1/loans/{id}/return
2. System validates loan exists and is not already RETURNED
3. loan.returnDate = NOW(), loan.status = RETURNED; book_copy.status = AVAILABLE
4. If overdue: calculate overdue days, apply grace period, calculate fine, cap at max_fine, create Fine record (status = PENDING)
5. Check reservation queue: if next PENDING reservation exists — promote to READY, set expiry = NOW() + 3 days, set book_copy.status = RESERVED, create RESERVATION_READY notification
6. Return updated LoanResponse

Loan Renewal Process

- Member/Librarian/Admin calls POST /api/v1/loans/{id}/renew
- Validates loan.status = ACTIVE and renewal_count < policy.maxRenewals
- Checks no PENDING reservations for this book (blocks renewal if queue exists)
- loan.dueDate += policy.maxLoanDays; loan.renewalCount++

- Returns updated LoanResponse

19. FINE CALCULATION SYSTEM

Fine Formula: overdue_days = return_date – due_date | chargeable_days = MAX(0, overdue_days – grace_period) | fine = MIN(chargeable_days × daily_rate, max_fine)

Membership	Daily Rate	Max Fine	Grace Period	Max Loan Days
STANDARD	Rs 5.00/day	Rs 500.00	1 day	14 days
PREMIUM	Rs 2.00/day	Rs 200.00	3 days	21 days
STUDENT	Rs 1.00/day	Rs 100.00	2 days	14 days

Worked Calculation Examples

Member Type	Days Overdue	Grace	Chargeable	Rate	Fine
STANDARD	5	1	4	Rs 5.00	Rs 20.00
PREMIUM (in grace)	3	3	0	Rs 2.00	Rs 0.00 (within grace)
STUDENT (capped)	120	2	118	Rs 1.00	Rs 100.00 (capped)

Fine Payment Workflow

```
POST /api/v1/fines/{id}/pay
Body: { "amount": 20.00, "paymentMethod": "CASH" }

Validation:
- Fine must not already be PAID or WAIVED
- Payment amount must not exceed outstanding balance

If paid_amount + payment = amount --> status = PAID
If paid_amount + payment < amount --> status = PARTIAL
```

20. USER / MEMBER MANAGEMENT SYSTEM

Membership Types and Privileges

Feature	STANDARD	PREMIUM	STUDENT
Max active loans	5	10	5
Loan duration	14 days	21 days	14 days
Max renewals	2	3	2
Grace period	1 day	3 days	2 days
Daily fine rate	Rs 5.00	Rs 2.00	Rs 1.00
Maximum fine	Rs 500.00	Rs 200.00	Rs 100.00

Member Status Lifecycle

```
ACTIVE --(suspend)--> SUSPENDED --(reactivate)--> ACTIVE
ACTIVE --(expire membership)--> EXPIRED --(renew)--> ACTIVE
```

Self-Service Capabilities (ROLE_MEMBER)

- View own profile and update personal details
- View loan history and currently active loans
- Renew own loans (subject to renewal rules)
- View and pay own outstanding fines
- Place and cancel book reservations
- View own notifications

21. DEFAULT LOGIN CREDENTIALS

Admin Account

Email	admin@library.com
Password	Admin@123
Role	ROLE_ADMIN
Employee Code	EMP-0001
Designation	Administrator

Sample Member Accounts (create via POST /api/v1/members)

Email	Password	Membership	Max Loans	Fine Rate
alice@example.com	Password@1	PREMIUM	10	Rs 2.00/day
bob@example.com	Password@1	STUDENT	5	Rs 1.00/day
carol@example.com	Password@1	STANDARD	5	Rs 5.00/day

Accessing Swagger UI

```
1. Open: http://localhost:8085/swagger-ui.html
2. Click Authorize button (top right)
3. Enter: Bearer <paste-your-access-token>
4. All authenticated endpoints become accessible
```

22. STEP-BY-STEP IMPLEMENTATION PROCESS

Phase 1: Environment Setup

- Install JDK 8 — set JAVA_HOME; verify: java -version
- Install MySQL 8 — set root password; verify: mysql -u root -p
- Install Apache Maven 3.9.6 — add to PATH; verify: mvn -version
- Install IntelliJ IDEA (recommended) or Eclipse

Phase 2: Database Setup

- Run: CREATE DATABASE IF NOT EXISTS library_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
- Flyway will automatically run V1__create_schema.sql and V2__seed_data.sql on first startup

Phase 3: Configure Application

- Edit src/main/resources/application.yml — set datasource URL, username, password
- Set JWT_SECRET environment variable to a strong secret (min 32 chars)
- Set server.port=8085 (or preferred port)

Phase 4: Build and Run

- Run: mvn clean install -DskipTests
- Run: java -jar target/library-management-system-1.0.0.jar
- Application starts, Flyway runs migrations, default admin is seeded

Phase 5: Testing

- Open Swagger UI at http://localhost:8085/swagger-ui.html
- Login with admin@library.com / Admin@123 to get JWT token
- Use Postman or Swagger to test all endpoints
- Create sample members (alice, bob, carol) and run loan/fine/reservation scenarios

23. ADVANTAGES OF THE PROJECT

Advantage	Description
Stateless Scalability	JWT-based auth means no server-side session; add more instances behind a load balancer without shared state
Tier-Based Fine Engine	Membership-specific daily rates, grace periods, and caps ensure fair, consistent, and transparent fine calculation
Automated Notifications	@Scheduled background jobs remove the need for manual reminders, reducing overdue rates
Priority Reservation Queue	First-come-first-served queue with 3-day collection window ensures fair book allocation
In-Memory Caching	@Cacheable on book listings dramatically reduces DB load for the most frequently accessed endpoint
Flyway Migrations	Version-controlled schema ensures identical, reproducible deployments across dev/test/production
Swagger API Docs	Any frontend developer or third-party integrator can explore and test the API without reading source code
Global Exception Handler	Consistent JSON error responses with HTTP status codes make the API predictable for all clients
BCrypt Password Security	Cost factor 12 hashing makes brute-force attacks computationally infeasible
DTO Pattern	Decouples the API contract from the database schema; prevents accidental exposure of sensitive fields

24. FUTURE ENHANCEMENTS

- **Redis Distributed Cache** — Replace the in-memory cache with Redis to support multi-instance deployments sharing a single cache.
- **Email / SMS Notifications** — Integrate Spring Mail (SMTP) and Twilio for real email/SMS delivery of due reminders and reservation alerts.
- **Payment Gateway Integration** — Integrate Razorpay or Paytm for online fine payments.
- **React / Angular Frontend** — Build a web-based admin dashboard and member portal consuming the existing REST API.
- **Android / iOS Mobile App** — A mobile app for members to search books, manage loans, pay fines, and place reservations.
- **E-Book Hosting** — Add a digital content module for PDF/EPUB hosting with read-online capability.
- **Barcode / QR Scanner Integration** — Enable librarians to scan barcodes via webcam or mobile camera for faster checkout.
- **Multi-Branch Support** — Extend the system to manage multiple library branches with inter-branch loan transfers.
- **OAuth2 / SSO** — Add Google/Microsoft SSO for institutional login without separate password management.
- **Read Replicas** — Direct all read queries to MySQL read replicas to handle high concurrent loads.
- **RabbitMQ / Kafka** — Replace @Scheduled notification delivery with an async message queue for reliability and scalability.
- **Elasticsearch Integration** — Full-text search across books, authors, and descriptions for faster and more relevant catalogue queries.

25. CONCLUSION

The **Library Management System** developed using **Java 8**, **Spring Boot 2.7.18**, and **MySQL 8** successfully addresses all the limitations of traditional manual library operations. The system provides a production-ready, cloud-deployable REST API that automates every aspect of library management — from book cataloguing and member registration to loan tracking, fine calculation, reservation queuing, and administrative reporting.

Key engineering achievements of this project include:

- A **stateless JWT security architecture** that scales horizontally without shared session state
- A **membership-tier fine policy engine** that is configurable via database records without code changes
- A **priority reservation queue** with automated notifications that eliminates the need for manual queue management
- **Flyway-managed schema migrations** ensuring identical, reproducible deployments across all environments
- A **fully documented Swagger API** enabling any frontend team to consume the API without source code access
- **@Cacheable optimization** on the most frequent read endpoint, reducing database load significantly under concurrent usage

This project demonstrates practical application of enterprise Java development patterns including RBAC, DTO mapping, layered architecture, transactional integrity, and RESTful API design — skills directly applicable in professional software development environments.

26. REFERENCES

[1] Spring Boot 2.7.x Reference Documentation — https://docs.spring.io/spring-boot/docs/2.7.x/reference/html/

[2] Spring Security 5.7 Reference — https://docs.spring.io/spring-security/reference/5.7/

[3] Spring Data JPA Reference — https://docs.spring.io/spring-data/jpa/docs/2.7.x/reference/html/

[4] JJWT (Java JWT) Library — https://github.com/jwtk/jjwt/tree/0.9.1

[5] Flyway Database Migrations — https://flywaydb.org/documentation/

[6] SpringDoc OpenAPI 3 — https://springdoc.org/v1/

[7] MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/refman/8.0/en/

[8] HikariCP Configuration Guide — https://github.com/brettwooldridge/HikariCP

[9] Hibernate 5 ORM Documentation — https://docs.jboss.org/hibernate/orm/5.6/userguide/html_single/

[10] MapStruct 1.5 Reference Guide — https://mapstruct.org/documentation/1.5/reference/html/

[11] Baeldung — Spring Boot REST API Tutorial — https://www.baeldung.com/rest-with-spring-series

[12] Martin Fowler — Patterns of Enterprise Application Architecture — Addison-Wesley, 2002

[13] Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship — Prentice Hall, 2008

27. VIVA QUESTIONS AND ANSWERS

Q1. What is the overall architecture of your Library Management System?

A: The system uses a three-tier REST architecture: (1) Client Layer — any HTTP client (browser, Postman, Swagger UI); (2) Business Logic Tier — Spring Boot 2.7.18 application with layered packages (Controller > Service > Repository); (3) Data Tier — MySQL 8 with 16 normalized tables. The security layer (JWT filter chain) sits between the client and controller. All layers communicate via well-defined interfaces and DTO objects.

Q2. How does JWT authentication work in your project?

A: On successful login, AuthServiceImpl issues two tokens using JJWT 0.9.1 with HS256 signing: an access token (15 min expiry) and a refresh token (7 days expiry). On every subsequent request, JwtAuthenticationFilter extracts the Bearer token from the Authorization header, validates it via JwtTokenProvider.validateToken(), extracts the email from the 'sub' claim, loads the user via CustomUserDetailsService, and sets the SecurityContext. When the access token expires, the client calls POST /auth/refresh with the refresh token to get a new access token — without re-entering credentials.

Q3. What is Role-Based Access Control and how is it implemented?

A: RBAC restricts system access based on the user's assigned role. Three roles exist: ROLE_ADMIN (full access), ROLE_LIBRARIAN (operations), ROLE_MEMBER (self-service). Roles are stored in the 'roles' table; permissions (BOOK_CREATE, LOAN_UPDATE, etc.) in 'permissions'; the 'role_permissions' join table maps roles to permissions. At runtime, @PreAuthorize annotations on service methods check roles before execution. @EnableMethodSecurity(prePostEnabled = true) in SecurityConfig activates this.

Q4. Explain the fine calculation formula.

A: Fine = MIN(daily_rate x MAX(0, overdue_days - grace_period), max_fine). The system first calculates overdue days (return_date - due_date). It then subtracts the membership-specific grace period. The remaining chargeable days are multiplied by the daily rate (Rs 5/Rs 2/Rs 1 for Standard/Premium/Student). The result is capped at the maximum fine (Rs 500/Rs 200/Rs 100). This logic runs in LoanServiceImpl.createOverdueFine() inside the @Transactional returnBook() method.

Q5. How does the reservation queue work?

A: When a member calls POST /api/v1/reservations, a reservation record is created with status=PENDING and a queue_position (next integer for that book). When any book copy is returned (returnBook()), the system queries for the lowest queue_position PENDING reservation for that book. If found: reservation.status = READY, expiry_date = NOW() + 3 days, book_copy.status = RESERVED, and a RESERVATION_READY notification is created. The member has 3 days to collect; if they don't, the reservation expires and the next in queue is promoted.

Q6. What is @Transactional and why is it used on issueLoan and returnBook?

A: @Transactional marks a method to run inside a database transaction. If any exception occurs, the entire transaction rolls back. In issueLoan(), creating the loan record and updating book_copy.status to BORROWED must happen atomically — if the copy status update fails, the loan record must also be rolled back to avoid inconsistency. Similarly, returnBook() must atomically update the loan status, the copy status, create the fine, and update the reservation queue.

Q7. What is Spring Boot auto-configuration?

A: Spring Boot auto-configuration automatically configures Spring beans based on the classpath and application.yml properties. Adding spring-boot-starter-security is enough for Spring Boot to configure DelegatingFilterProxy, UserDetailsService, PasswordEncoder, and the security filter chain without any XML or boilerplate. Defaults are overridden by providing @Bean definitions in configuration classes (e.g., SecurityConfig, RedisConfig).

Q8. What is the DTO pattern and why not return JPA entities directly?

A: DTOs (Data Transfer Objects) are plain objects that carry data between layers. Returning JPA entities directly causes: (1) potential N+1 queries from lazy-loaded associations, (2) exposure of sensitive fields like password_hash to API clients, (3) tight coupling between the database schema and the API contract. DTOs like BookResponse, LoanResponse, and MemberResponse expose only the fields needed by clients.

Q9. What is Flyway and why use it?

A: Flyway is a database migration tool that applies versioned SQL scripts in order. Instead of manually running CREATE TABLE statements on each environment, Flyway tracks applied migrations in flyway_schema_history and applies only new ones. V1__create_schema.sql creates all 16 tables; V2__seed_data.sql seeds roles, permissions, fine policies, and the default admin. This ensures every environment (dev, test, production) has an identical, reproducible schema.

Q10. How does @Cacheable work and where did you use it?

A: @Cacheable caches the return value of a method. When called again with the same parameters, Spring returns the cached result without executing the method. It is used on BookServiceImpl.getAllBooks() to reduce database load for the most frequently accessed endpoint. The cache (named 'books') is cleared using @CacheEvict when books are added or updated, ensuring stale data is never served.

Q11. How would you scale this application to 10,000 concurrent users?

A: (1) Horizontal scaling — JWT is stateless, so multiple instances behind a load balancer are already supported. (2) Redis cache — replace in-memory cache with Redis for shared caching across instances. (3) Read replicas — direct read queries to MySQL read replicas. (4) Async notifications — use RabbitMQ/Kafka instead of @Scheduled for notification delivery. (5) Elasticsearch — for faster full-text catalogue searches. (6) Connection pool tuning — increase HikariCP pool size per instance.

Q12. What prevents a member with unpaid fines from borrowing?

A: In LoanServiceImpl.issueLoan(), before creating the loan, the system calls fineRepository.sumUnpaidByMember(member.getId()). This queries for all PENDING and PARTIAL fines for that member. If the total is greater than zero, a BusinessException is thrown with message 'Member has unpaid fines: {amount}' and the loan is blocked. The fine must be fully paid before any new book can be issued.

Q13. What is HikariCP and why configure it?

A: HikariCP is the default connection pool for Spring Boot. Instead of creating a new database connection per request (expensive), it maintains a pool of reusable connections. Configuration: maximum-pool-size=10 (max simultaneous DB connections), minimum-idle=2 (always keep 2 warm connections), connection-timeout=30000ms (throw if no connection available within 30 seconds). This makes the application efficient under concurrent load without exhausting MySQL connection limits.

Q14. What is the difference between @RestController and @Controller?

A: @Controller is the base MVC annotation that returns view names (HTML templates). @RestController = @Controller + @ResponseBody — every method return value is automatically serialized to JSON (or XML) and written directly to the HTTP response body. Since this is a pure REST API (no HTML views), all controllers use @RestController.

Q15. What is CustomUserDetailsService and why is it needed?

A: Spring Security's authentication requires a UserDetailsService that loads a UserDetails object (containing the password hash and granted authorities/roles) by username. CustomUserDetailsService first checks the 'staff' table (for admins and librarians), then falls back to the 'members' table. This allows a single /auth/login endpoint to authenticate all three user types transparently without separate login URLs.

End of Synopsis

Submitted By:	Student Signature:	Date:
_____	_____	_____
[Student Name] · [Roll Number]		May 21, 2026

Guide / Supervisor Signature:

_____

[Guide Name, Designation]

Department Seal:

This synopsis is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology / Master of Computer Applications / Bachelor of Computer Applications.